# Adaptive First-Order Methods for Logistic Objectives and Non-Convex Reference Landscapes

**Erion Krasniqi** **Denis Mustafa Xhabrahimi**
University of Basel
Continuous Optimization Project, Spring Semester 2026

## Abstract

Adaptive first-order methods modify the effective step size using the observed gradient history and are widely used in large-scale machine learning. This report compares stochastic gradient descent (SGD), Adagrad, AdaGrad-Norm, RMSprop, and Adam on two classification datasets, A9a and MNIST, using logistic and softmax cross-entropy objectives. We additionally evaluate logistic regression with the non-convex regularizer specified in the project description and include two controlled reference baselines: the Rosenbrock function, which exposes curved non-convex geometry, and full-batch L-BFGS as a quasi-Newton reference. Across the classification tasks, adaptive methods mainly improve the speed of optimization; final test accuracies are close across optimizers. The Rosenbrock benchmark makes Adam's advantage from momentum and adaptive coordinate scaling visible, while L-BFGS illustrates the behavior of a stronger deterministic full-batch method that is not directly comparable to mini-batch stochastic methods.

# 1 Introduction

Gradient-based optimization methods iterate

$$w_{t+1} = w_t - \eta_t g_t, \tag{1}$$

where $g_t$ is a gradient or stochastic gradient estimate. The practical difficulty is the choice of step size. A fixed learning rate can be too large in steep directions and too small in flat directions, especially in high-dimensional or sparse problems. Adaptive methods address this by scaling the update according to the trajectory of past gradients. This project studies the behavior of four adaptive methods, Adagrad, AdaGrad-Norm, RMSprop, and Adam, compared with SGD.

The project has two goals. First, we survey the mathematical update rules and qualitative differences between the methods. Second, we implement them in NumPy and compare their convergence on A9a and MNIST. Since the required logistic and softmax objectives are relatively well behaved, we also include a two-dimensional Rosenbrock benchmark to visualize a genuinely curved non-convex landscape. Finally, we add L-BFGS as a selected full-batch quasi-Newton reference, following the quasi-Newton framework surveyed by Xu and Zhang [Xu and Zhang, 2001].

# 2 Related Work and Methods

## 2.1 SGD

Stochastic gradient descent uses a single global learning rate:

$$w_{t+1} = w_t - \eta g_t. \tag{2}$$

It is cheap per iteration and memory efficient, but sensitive to the learning rate and unable to adapt per coordinate.

## 2.2 Adagrad

Adagrad accumulates squared gradients coordinate-wise [Duchi et al., 2011]:

$$G_t = G_{t-1} + g_t \odot g_t, \tag{3}$$

$$w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{G_t} + \epsilon}. \tag{4}$$

Coordinates with large historical gradients receive smaller effective steps. This is useful for sparse features but the accumulated denominator can grow monotonically, causing steps to decay.

## 2.3 AdaGrad-Norm

AdaGrad-Norm replaces coordinate-wise accumulation by a single scalar accumulator [Ward et al., 2020]:

$$b_t^2 = b_{t-1}^2 + \|g_t\|^2, \tag{5}$$

$$w_{t+1} = w_t - \frac{\eta}{b_t + \epsilon} g_t. \tag{6}$$

It retains adaptive scaling and admits sharp convergence guarantees over non-convex landscapes, but lacks per-coordinate adaptation.

## 2.4 RMSprop

RMSprop uses an exponential moving average of squared gradients:

$$v_t = \rho v_{t-1} + (1-\rho) g_t \odot g_t, \tag{7}$$

$$w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}. \tag{8}$$

Unlike Adagrad, it forgets old gradients and can react faster when the local geometry changes. The method is widely used but is mostly heuristic from a convergence-theory perspective [Ruder, 2016].

### 2.5 Adam

Adam combines momentum with RMSprop-style scaling [Kingma and Ba, 2014]:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{9}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \odot g_t. \tag{10}$$

The bias-corrected moments are

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{11}$$

and the update is

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \tag{12}$$

The first moment smooths the direction, while the second moment rescales coordinates. Reddi et al. [Reddi et al., 2018] showed that Adam can fail to converge in certain settings and proposed AMSGrad as a fix.

### 2.6 L-BFGS Reference

Quasi-Newton methods approximate curvature information using gradients and function values. In the classical BFGS form, the approximation $B_k$ to the Hessian is updated so that it satisfies the secant equation

$$B_{k+1} s_k = y_k, \qquad s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k). \tag{13}$$

This condition incorporates local curvature without explicitly forming the true Hessian. Limited-memory BFGS stores only a small number of recent $(s_k, y_k)$ pairs, which makes it practical for larger models while preserving the main quasi-Newton idea. We use L-BFGS-B from SciPy as a full-batch reference method for selected objectives. Because each L-BFGS iteration uses full objective and gradient evaluations, it is not treated as a mini-batch competitor; instead it contextualizes the stochastic first-order results.

## 3 Experimental Setup

### 3.1 Datasets

We use two LIBSVM datasets. A9a is a binary classification problem with 32,561 training samples and 123 sparse features; labels are represented as $y_i \in \{-1, +1\}$. MNIST is a ten-class classification problem with 60,000 training samples and 780 features in the LIBSVM version used here.

### 3.2 Objectives

For A9a, the convex logistic objective is

$$L(w) = \frac{1}{n} \sum_{i=1}^{n} \log(1 + \exp(-y_i x_i^\top w)). \tag{14}$$

We also evaluate the non-convex regularized objective from Reddi et al. [Reddi et al., 2016]:

$$L_{\mathrm{nc}}(w) = L(w) + \lambda \sum_j \frac{\alpha w_j^2}{1 + \alpha w_j^2}, \qquad \lambda = 10^{-2}, \quad \alpha = 1. \tag{15}$$

For MNIST, we use linear softmax regression with cross-entropy loss.

### 3.3 Gradients

For binary logistic regression, with $z_i = y_i x_i^\top w$, the gradient of the average loss is

$$\nabla L(w) = -\frac{1}{n} \sum_{i=1}^{n} y_i x_i \, \sigma(-z_i), \qquad \sigma(t) = \frac{1}{1 + \exp(-t)}. \tag{16}$$

The non-convex regularizer is separable across coordinates. For

$$r_j(w_j) = \frac{\alpha w_j^2}{1 + \alpha w_j^2}, \tag{17}$$

the derivative is

$$r_j'(w_j) = \frac{2\alpha w_j}{(1 + \alpha w_j^2)^2}. \tag{18}$$

This derivative is linear near zero but decays for large $|w_j|$, which makes the regularization term non-convex and qualitatively different from standard $\ell_2$ regularization. For MNIST, if $P = \mathrm{softmax}(XW)$ and $Y$ is the one-hot label matrix, then the full gradient is $X^\top(P - Y)/n$.

### 3.4 Reference Objective

To isolate non-convex geometry, we use the Rosenbrock function [Rosenbrock, 1960]

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \tag{19}$$

which has a narrow curved valley and global minimum at $(1, 1)$. This benchmark is not an additional dataset experiment; it is a controlled reference for optimizer trajectories.

### 3.5 Protocol

All stochastic optimizers are implemented from scratch in NumPy. We use mini-batches, report training loss and test accuracy, and average stochastic runs over three seeds. A9a is trained with batch size $128$ for 10 epochs, while MNIST is trained with batch size $256$ for 15 epochs. Hyperparameters are fixed after preliminary tuning: on A9a, SGD and Adagrad use $\eta = 10^{-1}$, AdaGrad-Norm uses $\eta = 1$, RMSprop uses $\eta = 10^{-2}$, and Adam uses $\eta = 5 \cdot 10^{-3}$. On MNIST, the corresponding values are $5 \cdot 10^{-2}$, $10^{-1}$, $5 \cdot 10^{-1}$, $10^{-3}$, and $10^{-3}$.

For the Rosenbrock reference, all methods start from the same initial point and are run for a fixed iteration budget so that trajectory differences are visible. For L-BFGS, we use deterministic full-batch objective and gradient evaluations. This distinction matters: one L-BFGS iteration can be substantially more expensive than one mini-batch first-order update, so the comparison should be read as a reference comparison rather than as a direct per-iteration race.

## 4 Results

### 4.1 Classification Objectives

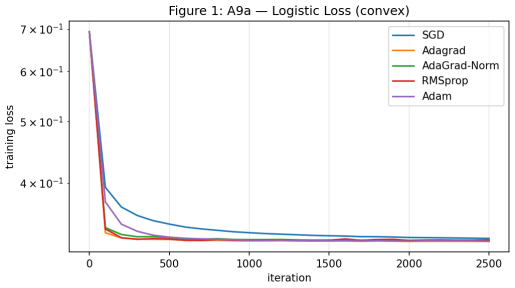
Figure 1 shows that adaptive methods decrease the loss faster in early iterations, while final accuracies are nearly identical ($84$–$85\%$). Adding the non-convex regularizer raises the objective but does not substantially change the optimizer ranking at $\lambda = 10^{-2}$.

On MNIST (Figure 2), Adagrad achieves the best final loss, with RMSprop and Adam close behind. Final accuracies differ only modestly, but adaptive methods reach good performance earlier.

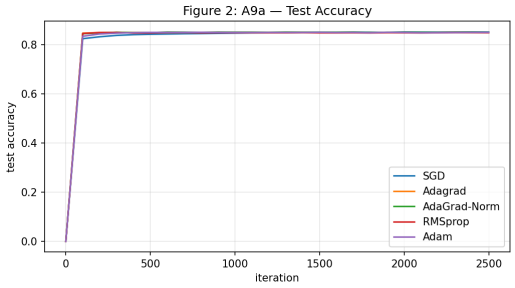
### 4.2 Robustness and Wall-Clock Time

Figure 3 shows that the per-step overhead of adaptive methods is small relative to their benefit from faster loss reduction. On both datasets, adaptive methods reach their final loss faster in wall-clock time.

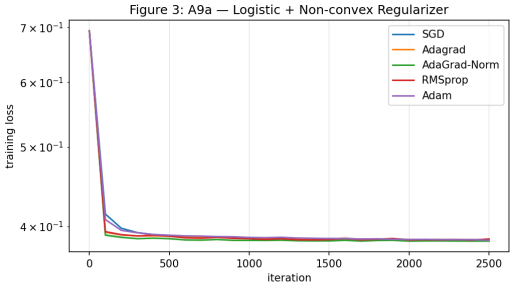
The learning-rate sweep in Figure 4 shows that SGD and AdaGrad-Norm are predictable over a broad range, while RMSprop becomes unstable for large rates. Adaptivity reduces, but does not remove, the need for tuning. The $\lambda$ sweep confirms that for small $\lambda$ the regularizer changes the objective only mildly; for larger $\lambda$ it dominates and accuracy drops, motivating our use of Rosenbrock as the clearer non-convex reference.



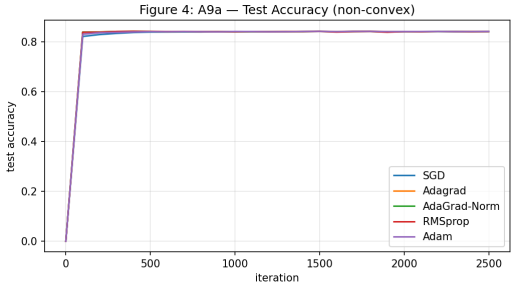

(a) A9a logistic loss




(b) A9a test accuracy

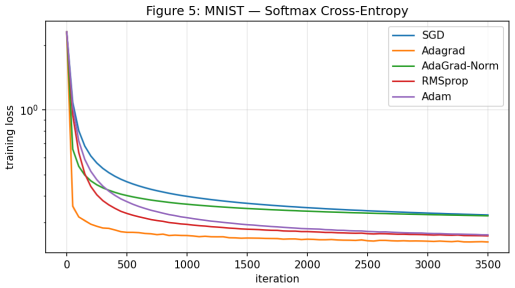



(c) A9a non-convex loss

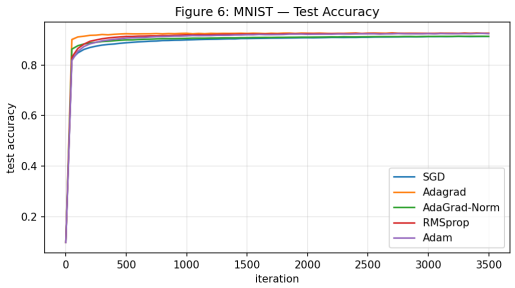



(d) A9a non-convex accuracy

Figure 1: A9a results. Adaptive methods reduce loss faster on convex logistic regression, while final test accuracy remains similar. The non-convex regularizer raises the objective value but does not substantially change the optimizer ranking at $\lambda = 10^{-2}$.




(a) MNIST softmax loss




(b) MNIST test accuracy

Figure 2: MNIST is higher-dimensional. Per-coordinate adaptation is more useful here: Adagrad reaches the lowest final loss, while RMSprop and Adam are close behind.

### 4.3 Gradient Norms and Reference Benchmarks

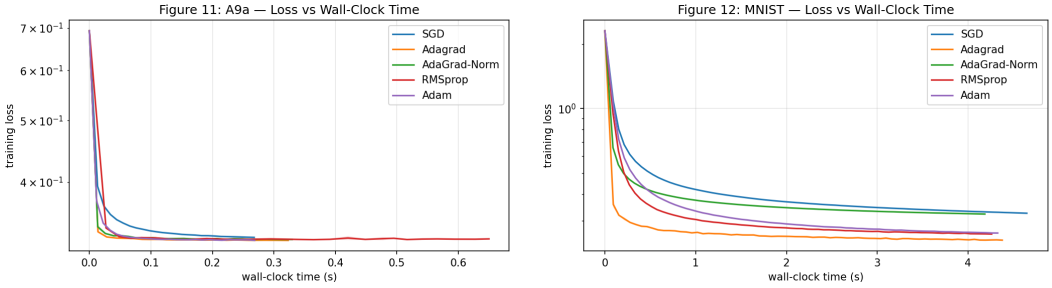
Gradient norms (Figure 5) show that adaptive methods normalize large early gradients and maintain progress as gradients shrink. On Rosenbrock, Adam's combination of momentum and coordinate-wise scaling is clearly visible: SGD requires many more iterations to follow the curved valley.

Figure 6 shows that L-BFGS obtains the best final loss on convex A9a and solves Rosenbrock in very few iterations, highlighting the difference between curvature-based deterministic methods and stochastic adaptive methods.

Table 1: Final training loss and test accuracy. Values for stochastic methods are means over three seeds. L-BFGS is a full-batch quasi-Newton reference and was only run on selected objectives.

| Optimizer | A9a loss | A9a acc | NC loss | NC acc | MNIST loss | MNIST acc |
|---|---|---|---|---|---|---|
| SGD | 0.3270 | 0.8516 | 0.3851 | 0.8425 | 0.3254 | 0.9134 |
| Adagrad | 0.3234 | 0.8495 | 0.3850 | 0.8425 | **0.2438** | **0.9266** |
| AdaGrad-Norm | 0.3244 | 0.8506 | **0.3838** | 0.8416 | 0.3225 | 0.9140 |
| RMSprop | 0.3252 | 0.8488 | 0.3861 | 0.8416 | 0.2603 | 0.9257 |
| Adam | 0.3234 | 0.8500 | 0.3847 | 0.8423 | 0.2631 | 0.9247 |
| L-BFGS* | **0.3226** | 0.8499 | 0.3835 | 0.8420 | – | – |

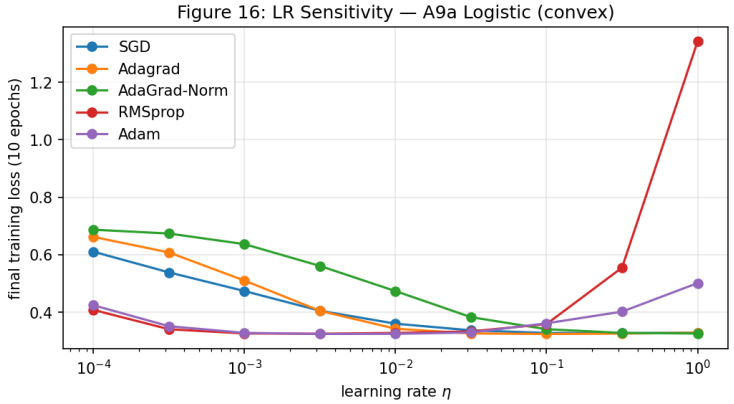
*Full-batch quasi-Newton reference; not directly comparable to mini-batch stochastic methods.



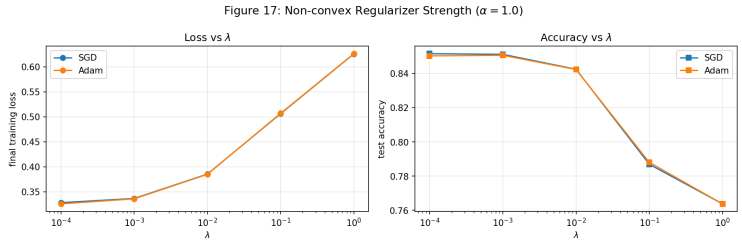

(a) A9a wall-clock

(b) MNIST wall-clock

Figure 3: Adaptive methods have slightly more per-step overhead, but they often reach low loss earlier in actual runtime because they need fewer effective optimization steps.

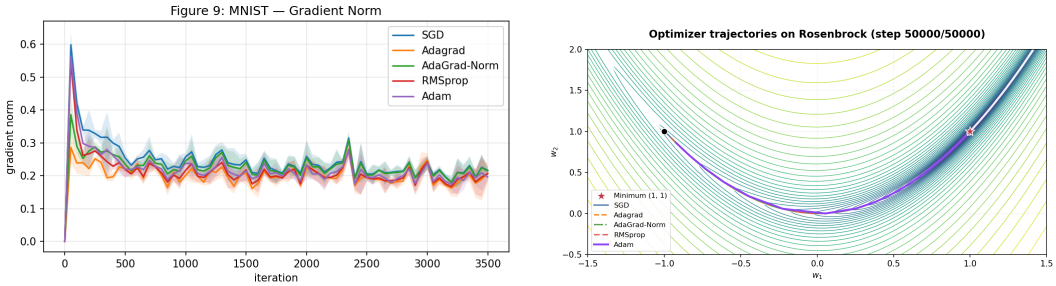



(a) Learning-rate sensitivity




(b) Regularization strength

Figure 4: Sensitivity experiments. The learning-rate sweep illustrates optimizer robustness, while the $\lambda$ sweep shows when the non-convex regularizer starts to dominate.




(a) MNIST gradient norms (b) Rosenbrock trajectories

Figure 5: Gradient norms explain the faster early progress of adaptive methods. Rosenbrock highlights Adam's advantage on a curved non-convex valley.

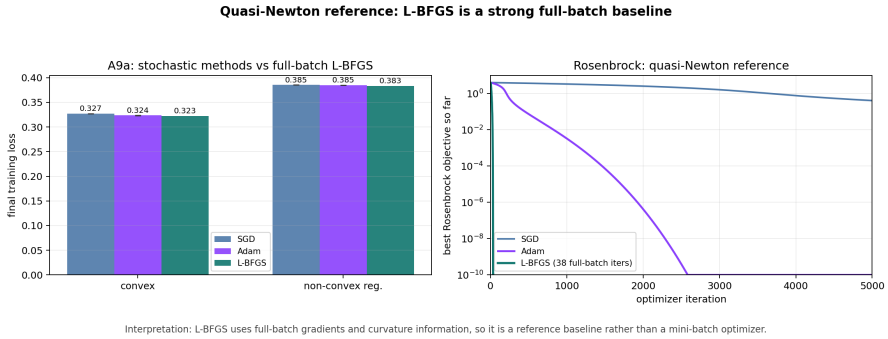



Figure 6: L-BFGS reference. On A9a, L-BFGS reaches comparable final losses. On Rosenbrock, it solves the benchmark in 38 full-batch iterations, illustrating why it should be interpreted as a separate deterministic reference rather than as a mini-batch competitor.

## 5 Discussion and Limitations

The main empirical result is consistent across datasets: adaptive methods improve convergence speed more than final classification accuracy. The strongest benefit appears in early loss reduction and in the ability to handle coordinate-wise gradient heterogeneity.

The comparison separates three notions that are often conflated: iteration efficiency, runtime efficiency, and final generalization. Adagrad and Adam can look strong per iteration because they rescale coordinates; in wall-clock time, this advantage remains only if the extra arithmetic is cheaper than the saved iterations. Final test accuracy is a different question: once the linear models are sufficiently optimized, all methods reach similar decision boundaries.

The non-convex regularizer from the project description has limited practical effect at $\lambda = 10^{-2}$. For this dataset and regularization strength, the landscape remains too well behaved to show a dramatic Adam advantage. Rosenbrock complements the dataset experiments by isolating the relevant geometry.

The quasi-Newton reference should be interpreted carefully. L-BFGS uses curvature information and full-batch gradients, so it is expected to perform very well on smooth objectives. Its role is not to replace adaptive first-order methods, but to contextualize stochastic results with a deterministic baseline.

Limitations include: manually tuned hyperparameters, L-BFGS evaluated only on selected objectives, and the use of linear models that do not fully capture deep non-convex loss surfaces.

## 6 Conclusion

We implemented and compared SGD, Adagrad, AdaGrad-Norm, RMSprop, and Adam on A9a and MNIST. Adaptive methods consistently accelerated early optimization, while final test accuracies remained similar. Adagrad performed especially well on MNIST, indicating that per-coordinate scaling is useful in higher-dimensional classification. The required non-convex regularized logistic objective did not strongly separate the optimizers at moderate regularization strength. A controlled Rosenbrock benchmark made Adam's advantage on curved non-convex geometry clearer, and L-BFGS provided a useful but separate quasi-Newton reference. Overall, the practical benefit of adaptive methods in our experiments is speed and robustness of optimization rather than substantially better final generalization.

## References


John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Sashank J. Reddi, Suvrit Sra, Barnabas Poczos, and Alexander J. Smola. Fast incremental method for smooth nonconvex optimization. *arXiv preprint arXiv:1603.06159*, 2016.

Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of adam and beyond. In *International Conference on Learning Representations*, 2018.

H. H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *The Computer Journal*, 3(3):175–184, 1960.

Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

Rachel Ward, Xiaoxia Wu, and Leon Bottou. Adagrad stepsizes: Sharp convergence over nonconvex landscapes. *Journal of Machine Learning Research*, 21(219):1–30, 2020.

Chengxian Xu and Jianzhong Zhang. A survey of quasi-newton equations and quasi-newton methods for optimization. *Annals of Operations Research*, 103:213–234, 2001.